МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет
по лабораторной работе № 8

по дисциплине «Машинно-зависимые языки программирования»
Вариант 4

Выполнил: ст. гр. ПС-13
Петялин А.Н.
Проверил: доцент, доцент кафедры ИиСП Баев А.А.

г. Йошкар-Ола
2025

Цель работы: узнать, что такое жидкокристаллические индикаторы, изучить их принцип работы.

Задания на лабораторную работу: написать программы с выводом информации на ЖКИ.

1. Теоретические сведения

ЖКИ WH1602. Данный индикатор позволяет одновременно отображать две строки по 16 символов, при этом каждый символ отображается с помощью матрицы точек 5x8.

Существует множество видов подобных индикаторов, при этом возможны вариации в размере матрицы символа, количестве строк и/или столбцов, но видов управляющих контроллеров гораздо меньше. Рассматриваемый индикатор имеет в своем составе контроллер HD44780. Данный контроллер широко распространен и применяется во многих индикаторах. Он фактически является промышленным стандартом и широко используется при производстве алфавитно-цифровых модулей.

Алфавитно-цифровые ЖКИ модули представляют собой удобное решение, позволяющее сэкономить время и ресурсы при разработке новых изделий, и при этом обеспечивают отображение большого объема информации при хорошей различимости и низком энергопотреблении.

Контроллер HD44780 потенциально может управлять двумя строками по 40 символов в каждой (для модулей с 4 строками по 40 символов одновременно используются сразу два контроллера) при матрице символа 5x7 точек.

Существует несколько стандартных конфигураций ЖКИ модулей (символов x строк): 8x2, 16x1, 16x2, 16x4, 20x1, 20x2, 24x2, 40x2, 40x4. Однако принципиальных ограничений на комбинации и количество отображаемых символов контроллер не накладывает – модуль может иметь любое количество символов от 1 до 80, хотя адресация в некоторых вариантах может оказаться весьма неудобной.

2. Практическая часть

Код программы:

```
#define F_CPU 1000000UL
#define D4 PINB0
#define D5 PINB1
#define D6 PINB2
#define D7 PINB3
#define RS PINB5
#define RW PINB6
#define E PINB7
#define CMD 0
#define DATA 1

#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>
uint8_t segments[] = {
        // GFEDCBA
        0b00111111, // 0 - A, B, C, D, E, F
        0b00000110, // 1 - B, C
        0b01011011, // 2 - A, B, D, E, G
        0b01001111, // 3 - A, B, C, D, G
        0b01100110, // 4 - B, C, F, G
        0b01101101, // 5 - A, C, D, F, G
        0b01111101, // 6 - A, C, D, E, F, G
        0b00000111, // 7 - A, B, C
        0b01111111, // 8 - A, B, C, D, E, F, G
        0b01101111, // 9 - A, B, C, D, F, G
};
void InitPorts(void);
void InitTimer1(void);
void Bin2Dec(uint16_t data);
void InitADC(void);
void InitUSART(void);
void SendString(char* buffer);
void SendChar(char symbol);
void InitLCD(void);
void LCD_Write(uint8_t type,char data);
char LCD_Read(void);
volatile uint8_t bcd_buffer[] = {0,0,0,0};
volatile uint16_t display_val = 0;
int main(void)
```

```
{
	InitPorts();
	InitTimer1();
	EIMSK |= (1 << INT0);  //Enable INT0
	EICRA |= (1 << ISC01); //Trigger on falling edge of INT0
	InitLCD();
	InitADC();
	InitUSART();
	sei();
	//global interrupt enable
	PORTB &= ~(1 << PINB0); //OE = low (active)
	SendString("Hello\r\n");
	LCD_Write(DATA,'H');
	LCD_Write(DATA,'e');
	LCD_Write(DATA,'l');
	LCD_Write(DATA,'l');
	LCD_Write(DATA,'o');
	LCD_Write(CMD,0x40|0x80); // ??????? ?? ?????? ??????
	LCD_Write(DATA,'V');
	LCD_Write(DATA,'a');
	LCD_Write(DATA,'l');
	LCD_Write(DATA,'u');
	LCD_Write(DATA,'e');
	LCD_Write(DATA,'=');
	LCD_Write(DATA,0x20); // ????? ???????
	while(1)
	{
		Bin2Dec(ADC);
		LCD_Write(CMD, 0x47 | 0x80); //????????? ??????? ?? 7-?
		//?????????? 2-? ??????
		LCD_Write(DATA, 0x30+bcd_buffer[3]);
		LCD_Write(DATA, 0x30+bcd_buffer[2]);
		LCD_Write(DATA, 0x30+bcd_buffer[1]);
		LCD_Write(DATA, 0x30+bcd_buffer[0]);
	}
}
//-------------------------------------------
ISR(TIMER1_COMPB_vect){
}
ISR(INT0_vect){
	SendString("Value = ");
	SendChar(0x30 + bcd_buffer[3]);
	SendChar(0x30 + bcd_buffer[2]);
	SendChar(0x30 + bcd_buffer[1]);
	SendChar(0x30 + bcd_buffer[0]);
```

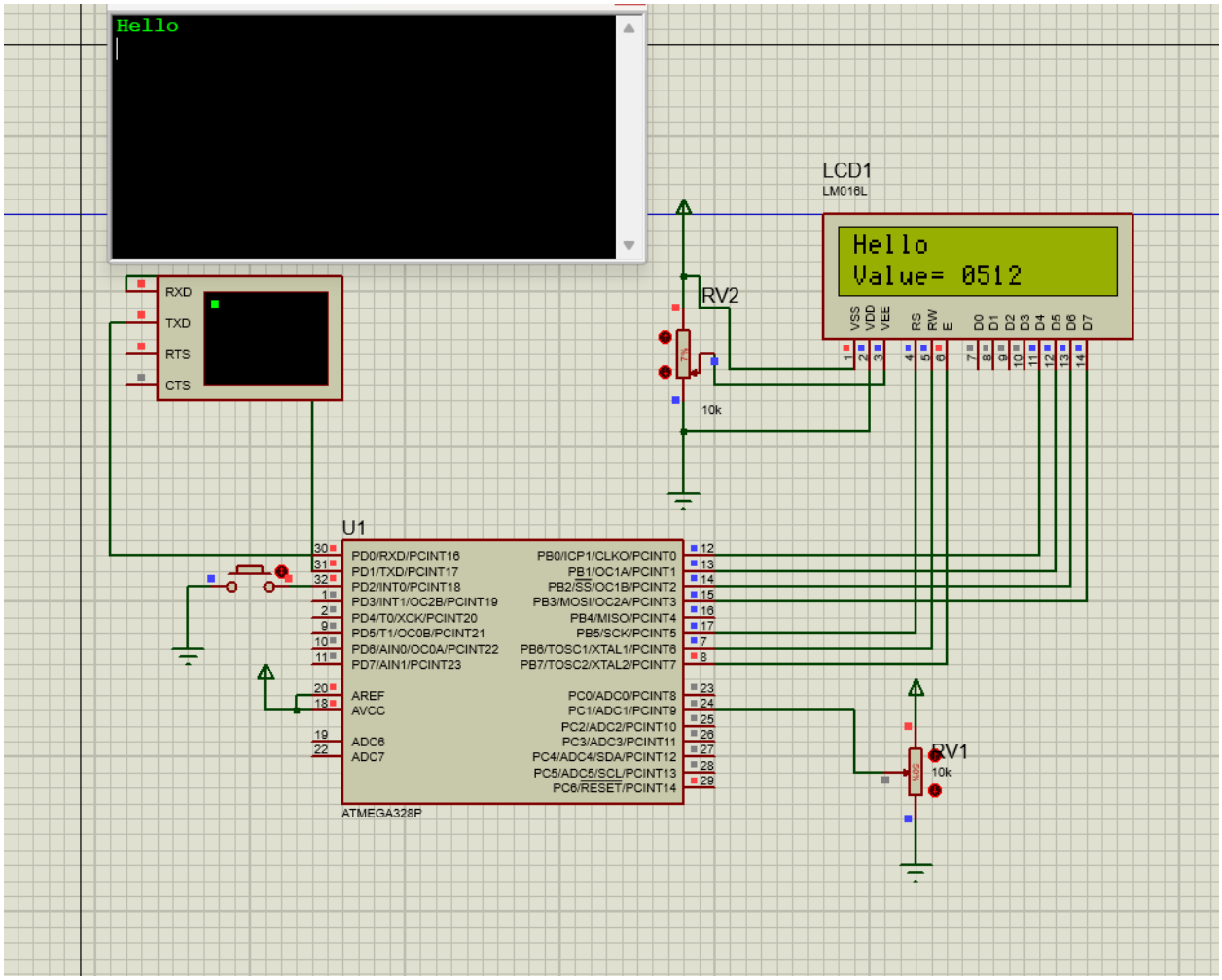
```
	SendString("\r\n");
}
ISR(USART_RX_vect){
	if(UDR0 == 0x20)
	{
		SendString("It was SPACE button\r\n");
	}
}
ISR(ADC_vect){
	display_val = ADC;
}
//-------------------------------------------
void InitPorts(void){
	DDRB = 0xFF;
	PORTB = 0;
	DDRD = (0<<PIND2);
	PORTD |= (1<<PIND2);
}
void InitTimer1( void){
	TCCR1A = 0;
	TCCR1B = (1<<CS11 | 1<<CS10 | 1<<WGM12);
	TCNT1 = 0;
	TIMSK1 |= (1<<OCIE1B);
	OCR1A = 1562;
	OCR1B = 1562;
}
void Bin2Dec(uint16_t data){
	bcd_buffer[3] = (uint8_t)(data/1000);
	data = data % 1000;
	bcd_buffer[2] = (uint8_t)(data/100);
	data = data % 100;
	bcd_buffer[1] = (uint8_t)(data/10);
	data = data % 10;
	bcd_buffer[0] = (uint8_t)(data);
}
void InitADC( void){
	ADMUX = (1<<MUX0);
	ADCSRB = (1<<ADTS2 | 1<<ADTS0);
	ADCSRA = (1<<ADEN | 1<<ADATE | 1<<ADIE);
}
void InitUSART(){
	UCSR0B = (1<<RXEN0 | 1<<TXEN0 | 1<<RXCIE0);
	UCSR0C = (1<<UCSZ01 | 1<<UCSZ00);
	UBRR0H = 0;
	UBRR0L = 0x0C;
```

```
}
void SendChar(char symbol){
	while (!(UCSR0A & (1<<UDRE0)));
	UDR0 = symbol;
}
void SendString(char * buffer){
	while(*buffer != 0){
		SendChar(*buffer++);
	}
}
void InitLCD(void)
{
	uint8_t BF = 0x80;
	_delay_ms(40);
	PORTB &= ~(1 << RS);
	PORTB = (0x30 >> PINB4);
	PORTB |= (1 << E);
	asm("nop");
	asm("nop");
	asm("nop");
	PORTB &= ~(1 << E);
	PORTB = 0;
	_delay_ms(5);
	PORTB &= ~(1 << RS);
	PORTB = (0x30 >> PINB4);
	PORTB |= (1 << E);
	asm("nop");
	asm("nop");
	asm("nop");
	PORTB &= ~(1 << E);
	PORTB = 0;
	_delay_us(150);
	PORTB &= ~(1 << RS);
	PORTB = (0x30 >> PINB4);
	PORTB |= (1 << E);
	asm("nop");
	asm("nop");
	asm("nop");
	PORTB &= ~(1 << E);
	PORTB = 0;
	_delay_ms(5);
	do
	{
		BF = (0x80 & LCD_Read());
	}while(BF == 0x80);
```

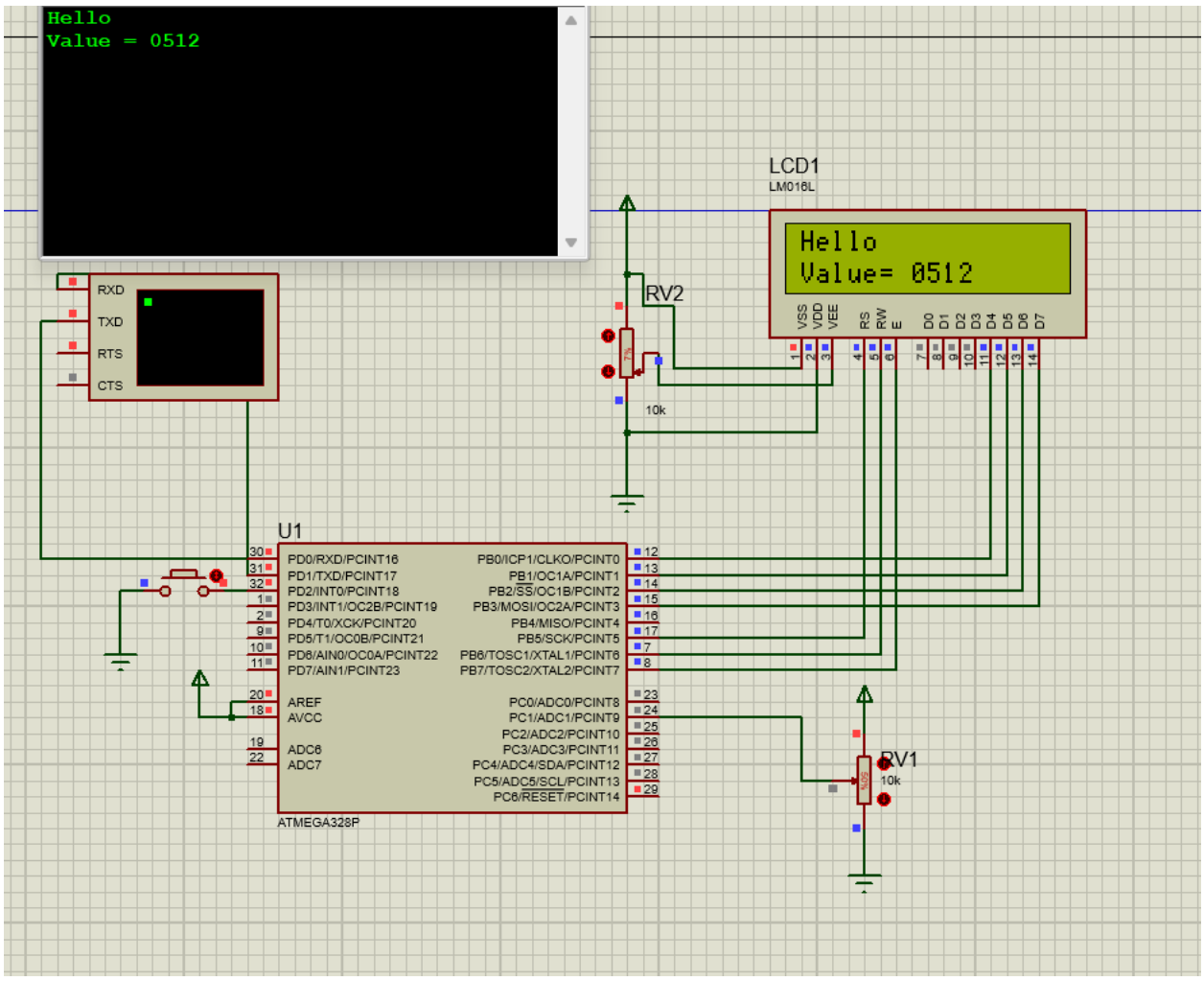
```
        PORTB &= ~(1 << RS);
        PORTB = (0x20 >> PINB4);
        PORTB |= (1 << E);
        asm("nop");
        asm("nop");
        asm("nop");
        PORTB &= ~(1 << E);
        PORTB = 0;
        do
        {
                BF = (0x80 & LCD_Read());
        }while(BF == 0x80);
        LCD_Write(CMD, 0x28);
        LCD_Write(CMD, 0x0C);
        LCD_Write(CMD, 0x06);
}
void LCD_Write(uint8_t type, char data){
        uint8_t BF = 0x80;
        do
        {
                BF = (0x80 & LCD_Read());
        }while(BF == 0x80);
        PORTB |= (type << RS);
        PORTB |= (1 << E);
        PORTB &= ~(0x0F);
        PORTB |= (0x0F & (data >> PINB4));
        PORTB &= ~(1 << E);
        asm("nop");
        asm("nop");
        asm("nop");
        PORTB |= (1 << E);
        PORTB &= ~(0x0F);
        PORTB |= (0x0F & data);
        PORTB &= ~(1 << E);
        PORTB = 0;
}
char LCD_Read(void)
{
        char retval = 0;
        PORTB &= ~(1 << RS);
        PORTB |= (1 << RW);
        DDRB &= ~(1 << D4 | 1 << D5 | 1 << D6 | 1 << D7);
        PORTB |= (1 << E);
        asm("nop");
        asm("nop");
```

```
        retval = (PINB & 0x0F) << 4;
        PORTB &= ~(1 << E);
        asm("nop");
        asm("nop");
        asm("nop");
        PORTB |= (1 << E);
        asm("nop");
        asm("nop");
        retval |= (PINB & 0x0F);
        PORTB &= ~(1 << E);
        DDRB |= (1 << D4 | 1 << D5 | 1 << D6 | 1 << D7);
        PORTB = 0;
        return retval;
}
```

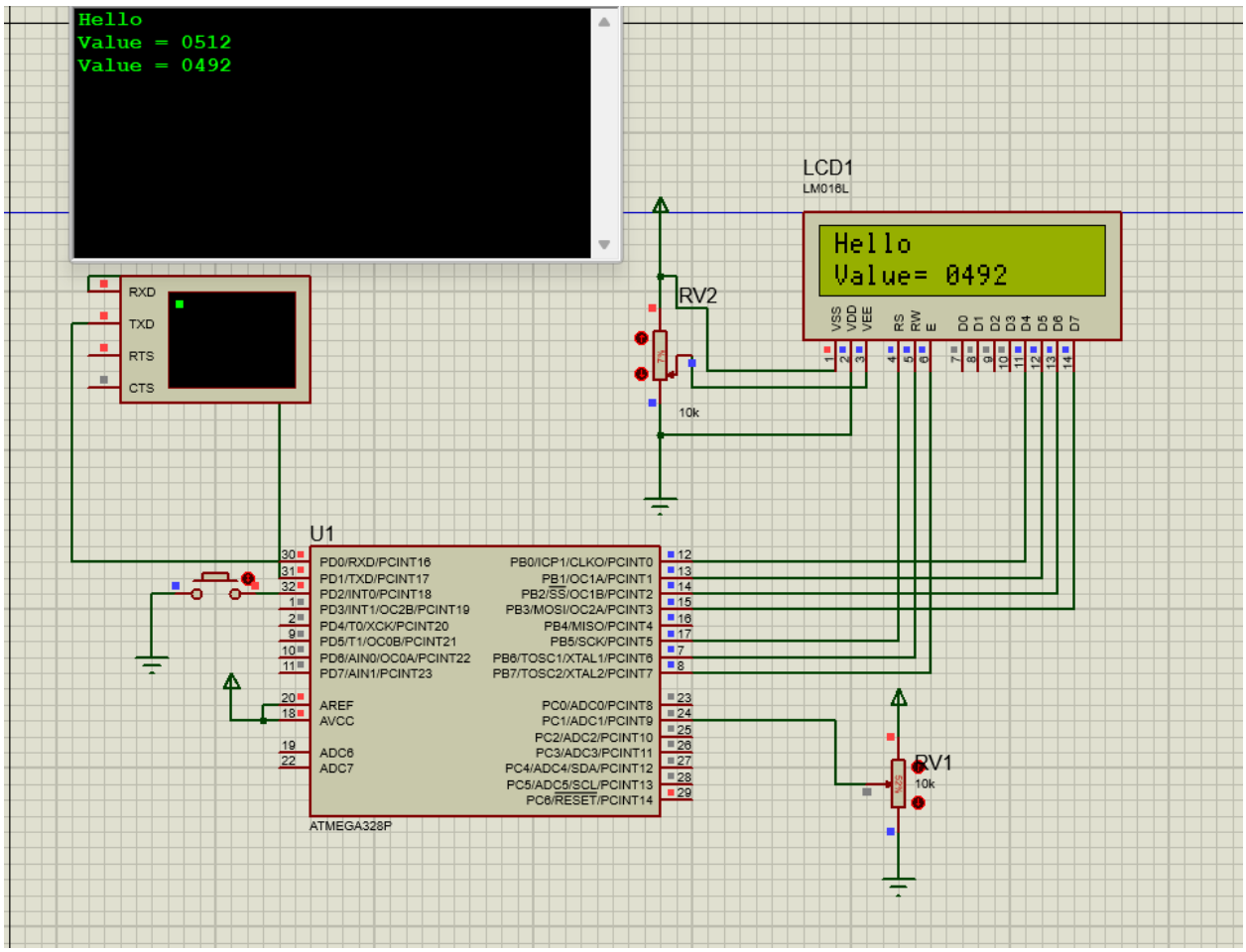
Схема в Proteus



После нажатия на кнопку в термина выводится нынешнее значение:



При смене значения на потенциометре меняется значение Value, оно также отобразится в терминале после нажатия кнопки.


Hello
Value = 0512
Value = 0492
LCD1
LM016L
Hello
Value= 0492
VSS VDD VEE RS RW E D0 D1 D2 D3 D4 D5 D6 D7
RV2
10k
RXD
TXD
RTS
CTS
U1
PD0/RXD/PCINT16
PD1/TXD/PCINT17
PD2/INT0/PCINT18
PD3/INT1/OC2B/PCINT19
PD4/T0/XCK/PCINT20
PD5/T1/OC0B/PCINT21
PD6/AIN0/OC0A/PCINT22
PD7/AIN1/PCINT23
AREF
AVCC
ADC6
ADC7
PB0/ICP1/CLKO/PCINT0
PB1/OC1A/PCINT1
PB2/SS/OC1B/PCINT2
PB3/MOSI/OC2A/PCINT3
PB4/MISO/PCINT4
PB5/SCK/PCINT5
PB6/TOSC1/XTAL1/PCINT6
PB7/TOSC2/XTAL2/PCINT7
PC0/ADC0/PCINT8
PC1/ADC1/PCINT9
PC2/ADC2/PCINT10
PC3/ADC3/PCINT11
PC4/ADC4/SDA/PCINT12
PC5/ADC5/SCL/PCINT13
PC6/RESET/PCINT14
ATMEGA328P
RV1
10k

Вопросы:

1. В чем заключаются недостатки светодиодных индикаторов?

К недостаткам относятся высокие габариты и высокое энергопотребления, из-за чего в компактных устройствах использовать светодиоды практически не имеет смысла.

2. Назовите преимущества и недостатки жидкокристаллических индикаторов.

ЖКИ достаточно компакты и дешевы, мало потребляют электроэнергии, однако являются достаточно хрупкими и подвержены влиянию низких температур.

3. Для чего в модули ЖКИ часто встраивают дополнительные контроллеры?

Управлять ЖКИ довольно тяжело, поэтому для упрощения работы выпускаются ЖКИ с встроенным контроллером, который берет на себя хранение и отображение символьной информации, поэтому управление подобным индикатором заключается в отправке данному контроллеру команд.

4. Назовите основные функциональные блоки контроллера HD44780.

Основными функциональными блоками являются:

1) Интерфейс управляющей системы
2) Регистр команд (IR)
3) Регистр данных (DR)
4) Видеопамять (DRAM)
5) Счетчик адреса памяти (AC)
6) Флаг занятости контроллера (BF)
7) Устройство исполнения команд
8) ОЗУ знакогенератора (CGRAM)

5. Каким образом происходит вывод необходимых символов на ЖКИ?

При отправке в ЖКИ кода символа он отображается в том месте экрана, на которое указывает счетчик АС, – после инициализации он указывает на первый символ первой строки.

После записи символа значение счетчика автоматически увеличивается, и он начинает указывать на второй символ. При записи последнего символа первой строки АС автоматически начинает указывать на первый символ второй.

Для непосредственного указания, какой символ требуется переписать, управляющий контроллер отправляет команду, содержащую новый адрес АС. Так, адрес первого символа первой строки – 0x00.

Однако адреса второй строки, вне зависимости от количества знакомест в строке ЖКИ, начинаются с адреса 0x40.

6. Вам необходимо отобразить на середине первой строки индикатора символ «Ы». Опишите порядок действий.

Для того, чтобы отобразить символ «Ы» на середине первой строки, сначала выясняем, какая таблица символов используется(нам нужна русская модификация таблицы), после этого нужно записать код символа 0x9B в регистр DR,установить курсор в нужную позицию, т.е. присвоить АС нужное значение 0x04.

Выводы

Я изучил принцип работы ЖКИ и научился составлять программы для вывода информации на него.